

Experiment No. 3

Aim: Build a SQLite-based app to store and retrieve student records.

Apparatus / Software Required :

Hardware	- Laptop(ASUS Vivobook 16x) - Minimum 8 GB RAM - AMD Ryzen 5 5600H with Radeon Graphics Processor - Internet Connection
Software	- Windows 11 Operating System - Android Studio (Quail) - Java Development Kit (JDK) - Android SDK - Android Emulator (Pixel 7) - Gradle Build System

Theory

SQLite is a lightweight local database used in Android applications to store data directly on the device. It does not require a separate database server.

In this application, SQLite stores **student name, roll number, course and marks**. The app allows users to **add, retrieve and delete** student records.

A database helper class manages the SQLite database and performs operations such as :-

- **Insert:** Add a new student's information into the database.
- **Retrieve:** Read and display saved student records.
- **Delete:** Remove a selected student record from the database.
- **Count:** Display the total number of students stored in the database.

The StudentVault app provides simple screens for adding and viewing student records, with a confirmation dialog before deletion.

Implementation Steps:

1. Open **Android Studio** and create a new Android project.
2. Create the required activities and XML layouts for the StudentVault application.
3. Design the **Main Dashboard** containing:
 - Total Students
 - Add Student

- View Records
 - Recent Students section
4. Create the **Add New Student** screen.
 5. Add input fields for:
 - Student Name
 - Roll Number
 - Course
 - Marks
 6. Add a course selection option containing courses such as B.Tech AIDS, B.Tech CSE, B.Tech IT and other courses.
 7. Create a SQLite database helper class to create and manage the student database.
 8. Create a student table in SQLite for storing the entered information.
 9. Implement the **Save Student** button to insert the entered student details into SQLite.
 10. Implement the **View Records** option to retrieve the stored student information from the database.
 11. Display the saved student records along with the total number of students.
 12. Add a **Delete** button for removing a selected student record.
 13. Display a confirmation dialog before deleting the record.
 14. After deletion, refresh the records screen and update the total student count.
 15. Run the application on the **Pixel 7 Android Emulator** and test adding, viewing and deleting student records.

Files Used and Code Changes

File Name	Purpose / Code Changes
MainActivity.java	Handles the StudentVault dashboard, student count and navigation to other screens.
AddStudentActivity.java	Takes student details, handles course selection and saves the record.
ViewStudentsActivity.java	Retrieves and displays saved student records and provides the delete option.
DatabaseHelper.java	Creates and manages the SQLite database and performs insert, retrieve and delete operations.
activity_main.xml	Defines the StudentVault dashboard interface.
activity_add_student.xml	Defines the Add New Student form interface.
activity_view_students.xml	Defines the Student Records interface.

Components Used

The following Android components were used in the application:

1. **TextView** - Used to display headings, labels and student information.
2. **EditText** - Used to enter student name, roll number and marks.
3. **Button** - Used for actions such as Add Student, Save, View and Delete.
4. **Spinner / Dropdown** - Used for selecting the student's course.
5. **LinearLayout** - Used to arrange the UI components.
6. **ScrollView** - Used to make the screen scrollable.
7. **CardView** - Used to display student records and dashboard information.
8. **AlertDialog** - Used to show delete confirmation.
9. **Toast** - Used to display short messages to the user.
10. **SQLiteDatabase** - Used for storing and managing student records locally.

Important XML Attributes Used

XML Attribute	Purpose
android:layout_width	Defines the width of a UI component.
android:layout_height	Defines the height of a UI component.
android:text	Sets the text displayed on a TextView or Button.
android:textSize	Sets the size of the text.
android:textColor	Sets the color of the text.
android:background	Sets the background color or drawable of a component.
android:padding	Adds space inside a UI component.
android:layout_margin	Adds space around a UI component.
android:gravity	Controls the alignment of content inside a component.
android:orientation	Defines the direction of a LinearLayout.
android:hint	Displays guidance text inside an EditText.
android:layout_weight	Distributes available space between components.

Important Java Methods Used

1. **onCreate()**
Initializes the Activity when it is created and connects the Java code with the XML layout.
2. **findViewById()**
Used to access UI components from the XML layout.
3. **setOnClickListener()**
Handles button click events.

4. **getText()**
Reads the value entered in an EditText.
5. **toString()**
Converts the entered value into a String.
6. **trim()**
Removes unnecessary spaces from the beginning and end of input.

SQLite Database Methods

The database helper uses SQLite operations for managing student records.

1. **Insert operation**
Adds a new student record to the database.
2. **Retrieve operation**
Reads stored records from the database.
3. **Delete operation**
Removes the selected record.
4. **AlertDialog**
Used to ask the user for confirmation before deleting a student.
5. **Toast.makeText()**
Displays a short message such as successful save or delete.
6. **startActivity()**
Used for moving from one Activity to another.

SQLite Database Operations

The application performs the following database operations:

1. **Create Database**
A local SQLite database is created for storing student records.
2. **Create Table**
A student table is created to store information such as:
 1. Student ID
 2. Student Name
 3. Roll Number
 4. Course
 5. Marks
3. **Insert Record**
When the user clicks **Save Student**, the entered details are inserted into the SQLite database.
4. **Retrieve Records**
When the user selects **View Records**, the application reads the stored records from SQLite and displays them.
5. **Delete Record**

When the user selects **Delete**, the application first displays a confirmation dialog. After confirmation, the selected record is deleted.

6. Update Count

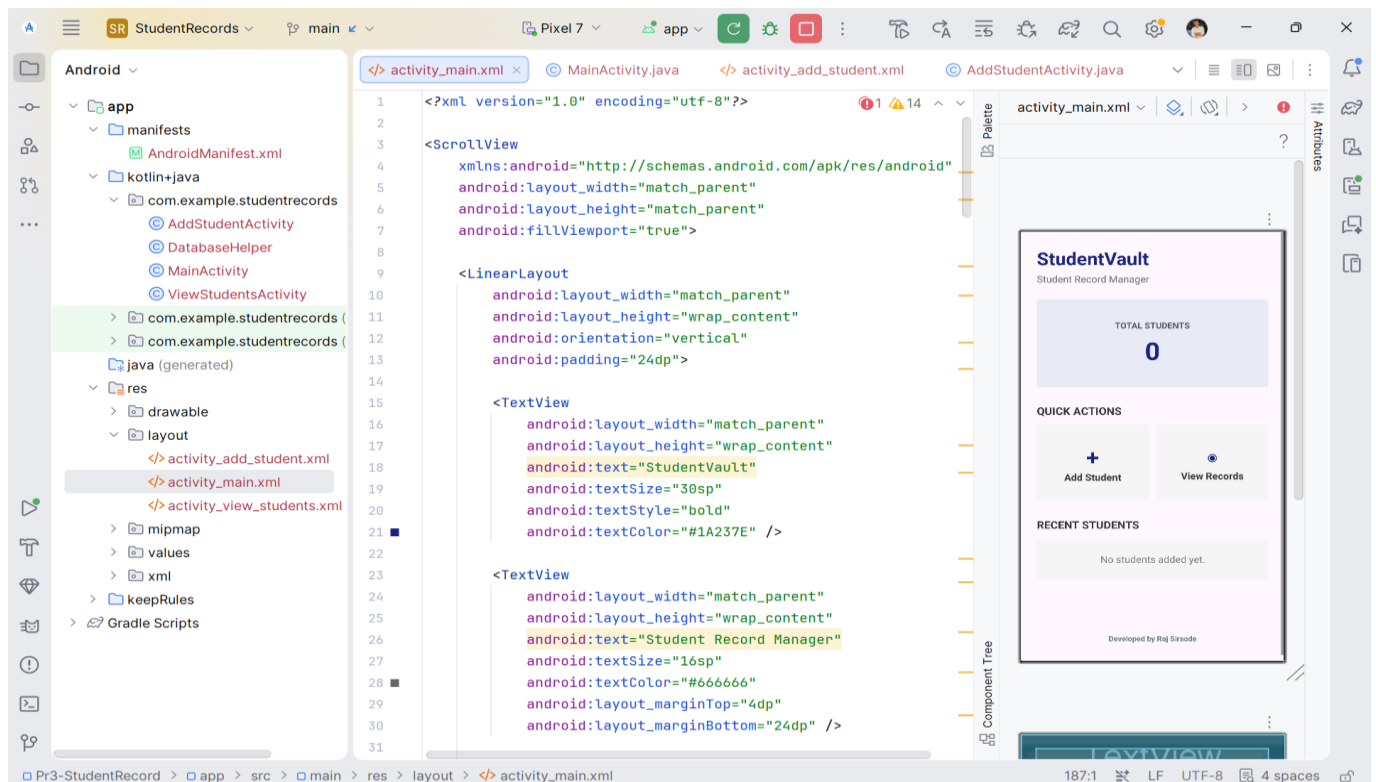
After adding or deleting records, the total number of students is updated on the application screen.

Errors Faced and Troubleshooting

Error / Problem	Solution / Troubleshooting
SQLite data not displaying	Checked the database retrieval method and ensured records were being read from the correct SQLite table.
Incorrect student input	Added input validation before saving the student record.
UI alignment issues	Adjusted padding, margin, layout_width and layout_height attributes.
Delete without confirmation	Added an AlertDialog to confirm before deleting a student record.
Student count not updating	Refreshed the database count after adding or deleting a record.

Output:

1. StudentVault Dashboard



StudentVault

Student Record Manager

TOTAL STUDENTS

0

QUICK ACTIONS



Add Student



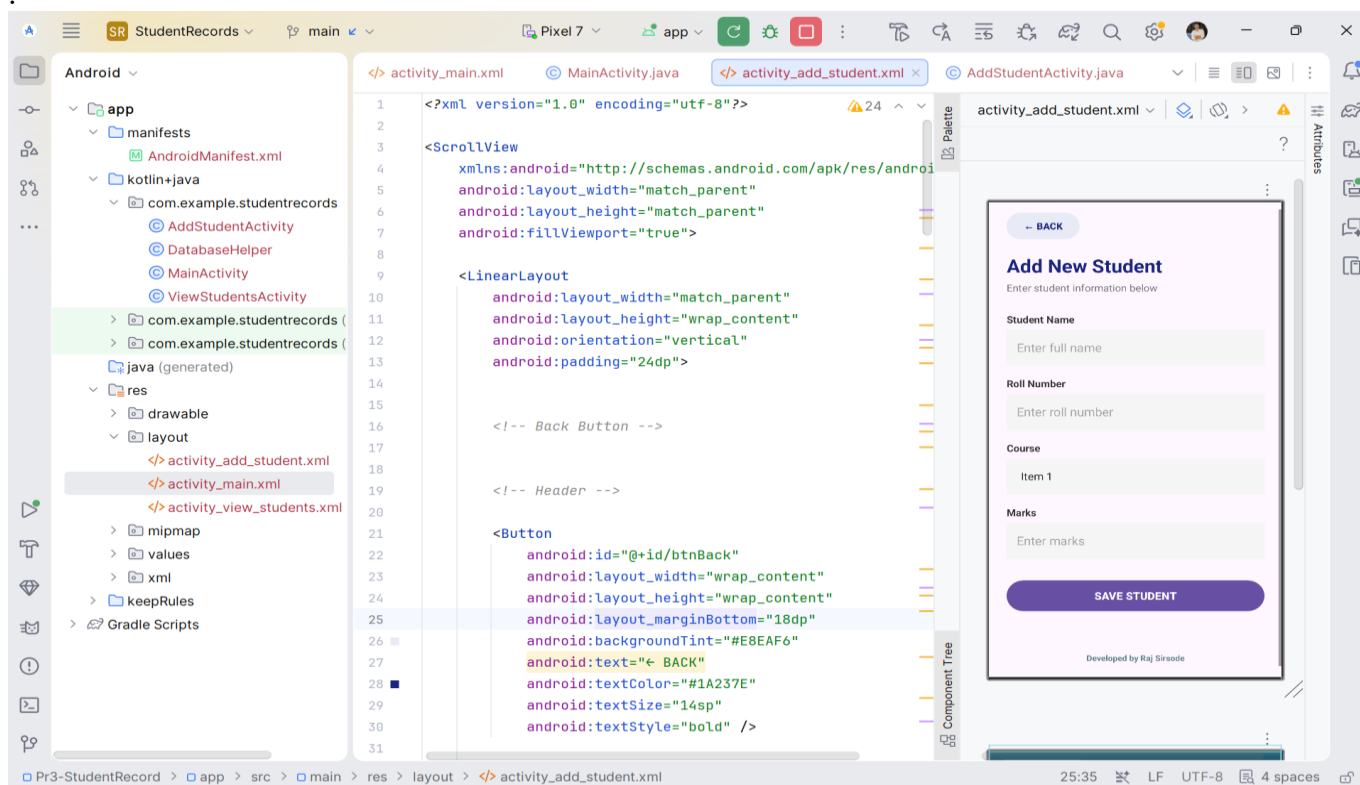
View Records

RECENT STUDENTS

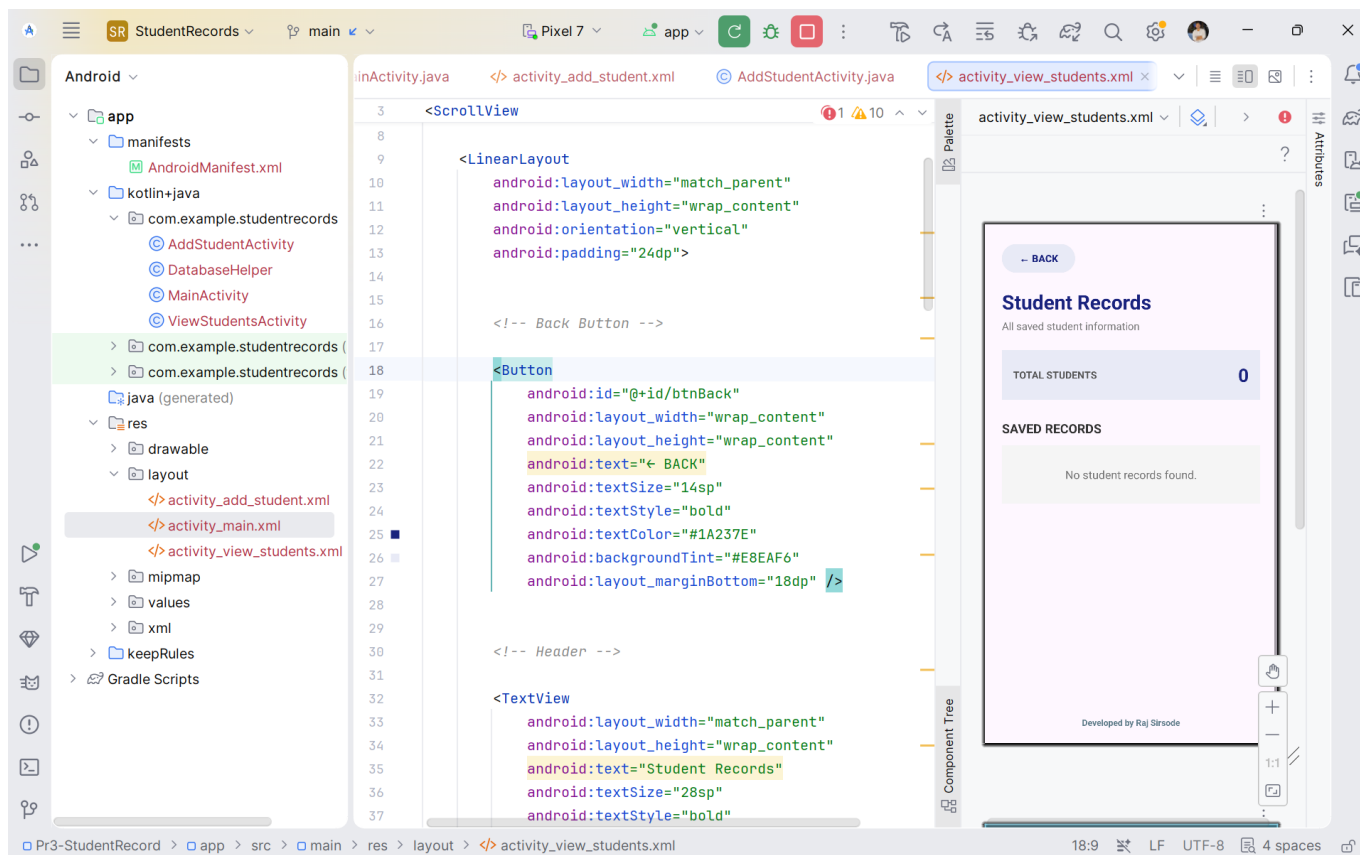
No students added yet.

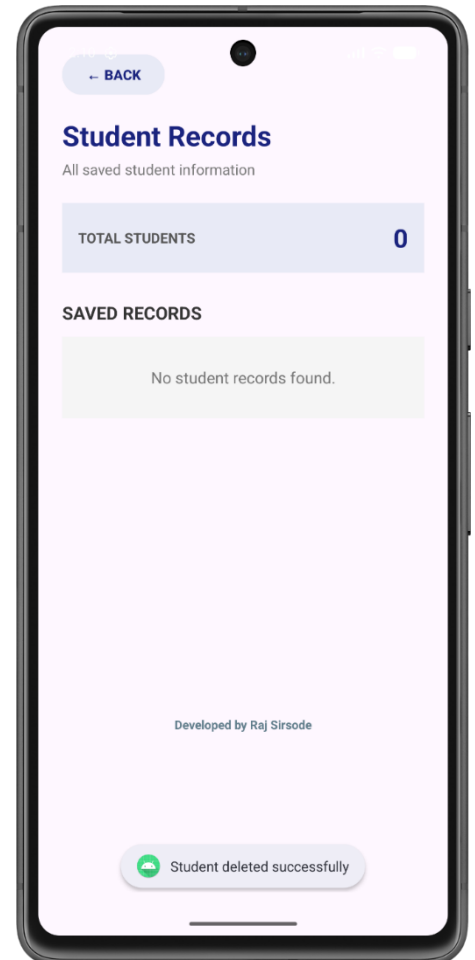
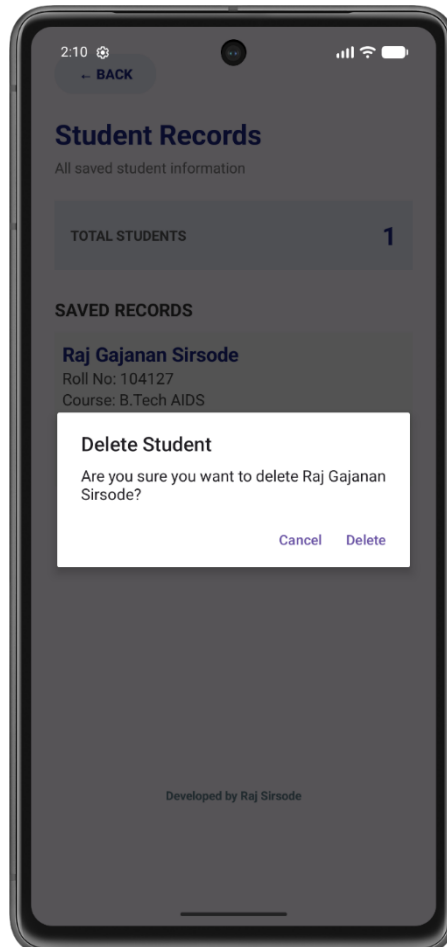
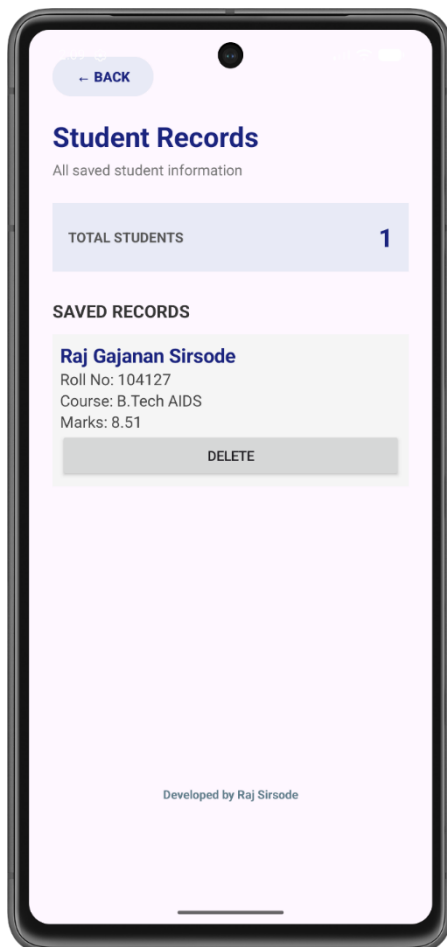
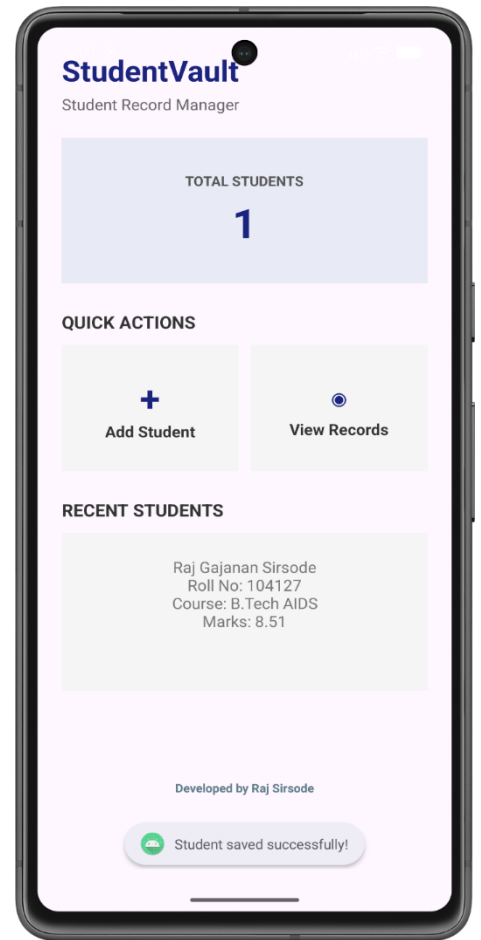
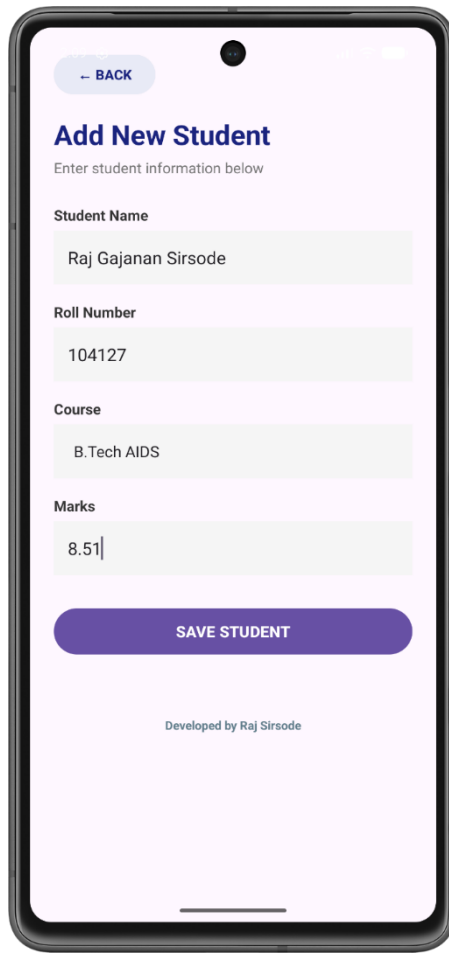
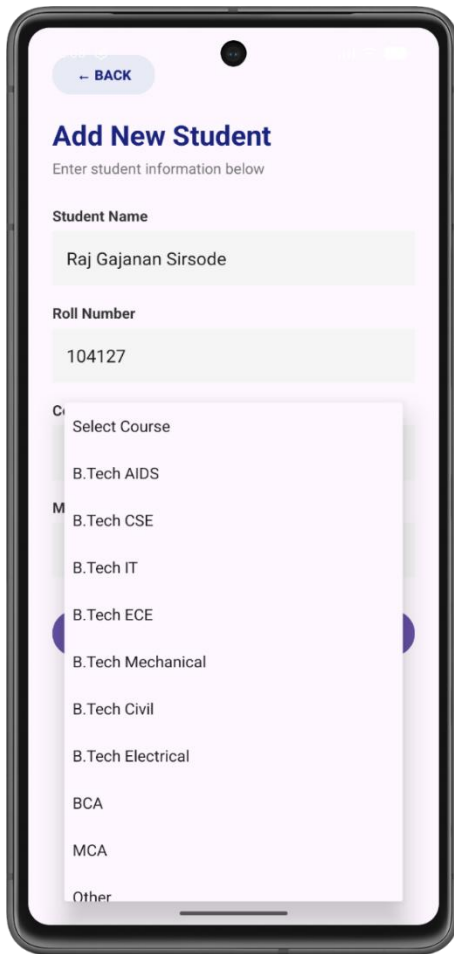
Developed by Raj Sirsode

2. Add New Student



3. Course Selection





Conclusion

The **StudentVault** application was successfully developed using **Android Studio and SQLite** to store and retrieve student records locally. The application allows the user to add student details such as name, roll number, course and marks, view the saved records and delete unwanted records.

A delete confirmation was also implemented to prevent accidental deletion. The application was tested on the **Pixel 7 Android Emulator**, and the main database operations such as inserting, retrieving and deleting student records were successfully demonstrated.

C (3)	A (3)	P (3)	T (9)	Sign